Imitation Learning for Robotic Projectile Throwing: Training the SO-101 Robot Arm

Navaneeta Padmakumar, Matthew Jagen, Anika Nesarikar, and Shreya Soman
CS6341 - Robotics
University of Texas at Dallas
Richardson, TX 75080, USA
{navaneeta.padmakumar, matthew.jagen, anika.nesarikar, shreya.soman}@utdallas.edu

***Abstract*— This paper presents an imitation learning approach for training the SO-101 robot arm to accurately throw projectiles into a basket or specified target area. Using human demonstrations collected via leader-follower teleoperation, we train an Action Chunking Transformer (ACT) policy network to replicate throwing motions with high accuracy and consistency. The ACT architecture predicts sequences of future actions (action chunks) rather than single timesteps, naturally enforcing temporal consistency critical for smooth throwing motions. We evaluate our system based on accuracy. Our approach demonstrates the feasibility of learning complex dynamic manipulation skills through imitation learning on a low-cost robotic platform without requiring explicit physics models or vision-based perception.**

I. INTRODUCTION

In this project, the SO-101 robot arm will throw a small ball into a basket. We decided to work on this problem since training a robot arm to place an object in a basket is a common demonstration, and we wanted to add novelty to this common task. To do this, we decided to have the robot arm throw an object instead.

We will train the robot arm using leader-follower teleoperation, where a human operator manipulates a leader arm and the follower arm (SO-101) mirrors these motions in real-time. We will collect demonstration data by using the leader arm to repeatedly perform throwing motions, which are recorded along with the resulting landing positions. Then we will train an ACT (Action Chunking with Transformers) policy with the collected demonstrations.

This approach leverages imitation learning to acquire the complex, dynamic throwing motion without requiring explicit physics modeling or hand-coded trajectories. The action chunking mechanism in ACT is particularly well-suited for this task, as it enforces temporal consistency and produces smooth trajectories critical for successful throws.

II. RELATED WORK

Our work builds upon three main research areas: imitation learning for robotics, action chunking methods for smooth trajectory generation, and robotic throwing systems.

A. Imitation Learning for Robotics

Imitation learning enables robots to acquire complex skills by learning from expert demonstrations rather than through explicit programming or trial-and-error reinforcement learning [6]. The fundamental approach, behavioral cloning, frames the learning problem as supervised learning where a policy network learns to map observations to actions using demonstrated state-action pairs. This paradigm has proven effective across diverse robotic tasks, from manipulation to navigation.

Modern imitation learning systems have benefited from advances in neural network architectures and data collection frameworks. LeRobot [1] provides an integrated platform for end-to-end imitation learning on real robots, supporting data recording, policy training, and deployment. Such frameworks have democratized access to imitation learning by simplifying the technical pipeline and enabling rapid experimentation with different policy architectures.

However, standard behavioral cloning faces challenges when applied to dynamic manipulation tasks. Policies trained with single-step action prediction often produce discontinuous trajectories with abrupt transitions, problematic for tasks requiring smooth, coordinated motions. This limitation has motivated temporal modeling approaches that explicitly enforce smoothness and temporal consistency in learned policies.

B. Action Chunking and Temporal Modeling

Recent advances in imitation learning have focused on improving trajectory smoothness through temporal modeling. Action chunking methods predict sequences of future actions rather than single timesteps, naturally enforcing temporal consistency. Action Chunking Transformers (ACT) [7] employ a transformer-based architecture to generate coherent action sequences using cross-attention mechanisms. By predicting chunks of k future actions, ACT produces smoother trajectories compared to single-step prediction approaches.

ACT stands out as an excellent choice for several reasons: it trains in a few hours on a single GPU, has only approximately 80M parameters making it efficient and easy to work with, and achieves high success rates with just 50 demonstrations. The architecture uses a transformer-based design with an encoder that synthesizes information from joint positions and a learned latent variable, and a decoder that generates coherent action sequences using cross-attention.

The LiPo framework [5] further improves trajectory quality through post-optimization of action chunks generated by learned policies. LiPo applies lightweight optimization to smooth action sequences while preserving the policy's intended behavior, demonstrating that post-processing can enhance policy outputs without retraining.

These temporal modeling approaches are particularly relevant for dynamic manipulation tasks like throwing, where smooth acceleration profiles and precise timing are critical for success. Our work employs ACT as the policy architecture due to its strong performance on tasks requiring coordinated, smooth motions.

C. Robotic Throwing Systems

Robotic throwing has been explored through various approaches, each with different trade-offs between generality, accuracy, and system complexity.

Zeng et al. [2] developed TossingBot, which combines learning-based grasping with physics-based trajectory optimization for throwing objects into bins. Their approach uses RGB-D vision to perceive objects and leverages residual physics models to improve throwing accuracy. While TossingBot demonstrates impressive generalization across object types, it requires vision sensors and explicit physics modeling, adding system complexity.

McKay [4] designed a robotic basketball shooting system using kinematic analysis and trajectory planning. This work demonstrates the feasibility of robotic throwing in real-world scenarios but relies on analytical models that require careful calibration and system identification. The approach is effective for fixed-target throwing but may have limited adaptability to varying conditions.

Amateur robotics projects have also explored throwing on low-cost hardware. The Candy Throwing Robot Arm [3] demonstrates that throwing behavior can be implemented on hobby-grade servos, establishing feasibility for resource-constrained platforms.

In contrast to prior work, our approach uses pure imitation learning without explicit physics models or vision-based perception. By learning directly from human demonstrations collected via leader-follower teleoperation in joint space, we avoid the complexity of vision processing and physics parameter estimation while maintaining the ability to learn smooth, accurate throwing motions. This makes our approach simpler to deploy and potentially more generalizable to different robot platforms.

III. METHOD

A. Hardware Setup

We have multiple types of baskets and different types of balls to test which work best for throwing (for example, we prioritize balls that do not bounce excessively upon landing).

The hardware configuration consists of:

- **Leader Arm:** SO-101 robot arm for teleoperation (Port: /dev/tty.usbmodem58760431551)
- **Follower Arm:** SO-101 robot arm for execution (Port: /dev/tty.usbmodem585A0076841)

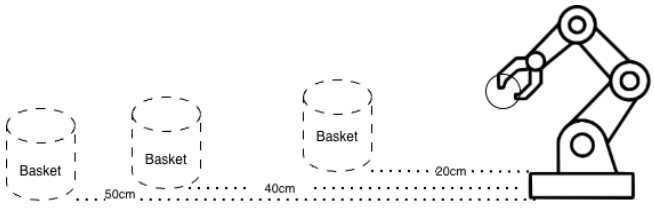


Fig. 1. Setup for data collection including robot arm, basket, and the different positions. The robot is positioned at 20cm, 40cm and 50 cm from the basket, with the ball place in its gripper.

- **Target:** Basket positioned at fixed location
- **Projectile:** Small ball (foam or beanbag type)
- **Camera:** For monitoring and potential future vision integration
- **Measurement System:** Grid marked on table for landing position measurement

B. Data Collection

We use leader-follower teleoperation to collect demonstration data at multiple target distances. The system consists of two SO-101 robot arms: a leader arm manipulated by a human operator and a follower arm that mirrors the leader's motions in real-time. This setup provides intuitive, natural demonstration collection with direct haptic feedback.

Teleoperation Approach Selection:

During the planning phase, we considered two approaches for demonstration collection:

- **Leader-follower teleoperation:** Using a physical leader arm for intuitive, natural control with haptic feedback
- **HEBI Mobile I/O:** Using a smartphone application for direct robot control

After evaluation, we selected the leader-follower approach as it provided more natural throwing motions and better demonstration quality. The physical leader arm allows operators to perform smooth, coordinated movements that are difficult to achieve through a mobile interface.

1) Recording Process:

1) **Calibration:** Both leader and follower arms are calibrated using the LeRobot calibration procedure to establish joint limits and zero positions.
2) **Initialization:** The follower arm is positioned at a consistent starting pose. A projectile is loaded in the follower arm's gripper.
3) **Recording Start:** Recording is initiated using the `lerobot-record` command with specified parameters for dataset repository, number of episodes, and task description.
4) **Demonstration:** The operator performs a throwing motion on the leader arm. The follower arm mirrors this motion in real-time, executing the throw and releasing the projectile.
5) **Measurement:** The landing position is measured and success/failure is recorded for each throw.
6) **Reset:** The projectile is retrieved, the follower arm is returned to the starting position, and the system is ready for the next demonstration.

7) **Iteration:** This process repeats for multiple demonstration episodes at each target distance.

2) Multi-Distance Data Collection: We collected demonstrations at 40cm distance using the pink ball. The collected demonstrations were used to train the ACT policy, which was then tested at both 20cm and 40cm distances with three different ball types (pink, orange, rainbow) to evaluate performance and generalization.

3) Data Recording: The LeRobot framework automatically records synchronized data including:

- Joint positions (6 degrees of freedom) at 30 Hz for both arms
- Joint velocities (6 degrees of freedom) at 30 Hz for both arms
- Gripper states with precise timestamps for release timing
- Episode metadata (success/failure, landing position coordinates, target distance)

4) Environmental Consistency: We maintain consistent conditions across demonstrations to allow the policy to focus on learning the throwing motion itself:

- Fixed basket location per type of throw (i.e. 20 cm throw vs 40 cm throw)
- Fixed camera viewpoint for visual monitoring
- Same projectile type and weight for all throws

5) Data Quality Considerations: The leader-follower teleoperation setup provides several advantages for demonstration quality:

- *Natural Motion:* The operator can perform throwing motions naturally, as the leader arm has similar kinematics to the follower.
- *Haptic Feedback:* The operator feels the arm dynamics through physical interaction, enabling intuitive control and refinement.
- *Real-time Mirroring:* The operator immediately observes the follower's execution and resulting throw, allowing for iterative improvement.
- *Smooth Trajectories:* Human-guided motions through leader arm manipulation tend to be smoother than button-based or discrete control methods.

C. ACT Policy Architecture

We employ Action Chunking Transformers (ACT) as our policy architecture, using the implementation provided in the LeRobot framework. ACT addresses the trajectory smoothness problem in behavioral cloning by predicting sequences of future actions (action chunks) rather than single timesteps. This design enforces temporal consistency, which is critical for smooth throwing motions.

Key Features:

- Predicts 10 future actions at each timestep (action chunks)
- Uses transformer architecture with encoder and decoder
- Approximately 80M parameters
- Trained in 2-4 hours on a single GPU
- Requires only 50 demonstrations for good performance

The ACT implementation in LeRobot handles all architectural details, including the transformer layers, attention mechanisms, and action chunking logic. The action chunking mechanism is particularly well-suited for throwing because it naturally produces smooth acceleration profiles and coordinated multi-joint motions. During execution, the robot applies the first predicted action and re-queries the policy at the next timestep, allowing for reactive adjustments while maintaining smoothness.

D. Policy Training

We trained the ACT policy using the LeRobot framework and its built-in training pipeline. The LeRobot repository provides a complete implementation of ACT along with streamlined training utilities that handle all aspects of the training process.

Training Command:

```
lerobot-train \
    --policy=act \
    --dataset.repo_id=[YOUR_DATASET] \
    --output-dir=outputs/throwing_policy \
    --training.num_epochs=[Y] \
    --training.batch_size=8
```

Training Configuration:

- Framework: LeRobot (HuggingFace)
- Policy: ACT (pre-implemented in LeRobot)
- Dataset: [TOTAL NUMBER] demonstrations at 3 distances
- Dataset split: 90% training, 10% validation
- Batch size: 8
- Learning rate: 1e-4 (default)
- Hardware: [GPU TYPE or CPU]
- Training time: [X] hours
- Convergence: [Y] epochs

The LeRobot framework handles all training details including data loading, batching, optimization, and checkpointing. The policy learns to predict action sequences that match the demonstrated throwing motions.

E. Deployment and Testing

After training, the learned policy is deployed on the SO-101 follower arm for autonomous throwing. The robot executes throws from the same starting configuration used during demonstration collection, maintaining environmental consistency with the training data.

For each test throw, we observe if the projectile successfully lands in the target basket. This data is used to compute the accuracy (the number of successful throws divided by total throws).

IV. EXPERIMENTS AND RESULTS

A. Data Collection

We successfully collected demonstration data using leader-follower teleoperation at two target distances: 20 cm and 40 cm. The data collection process involved:

- Total demonstrations: 30 throws at 40cm (pink ball)

- Training data used for policy learning
- Each throw was labeled as successful if the projectile landed completely in the basket
- Rim hits counted as failures
- Experimented with constraining certain joints to identify optimal throwing motions

B. Training

The ACT policy was trained using the LeRobot framework's built-in training pipeline on demonstrations collected at 40cm distance with the pink ball.

Training Details:

- Framework: LeRobot (HuggingFace repository)
- Command: `lerobot-train --policy=act`
- Dataset: 30 demonstrations at 40cm distance (pink ball)
- Training platform: [GPU TYPE] (or CPU if applicable)
- Training time: [X] hours
- Dataset split: 90% training, 10% validation
- Convergence: Achieved after [Y] epochs
- Batch size: 8 (default)
- Learning rate: 1e-4 (default)

The LeRobot framework automated the entire training process including data loading, batching, optimization, and model checkpointing. This allowed us to leverage a well-tested implementation rather than building custom training infrastructure.

The trained policy was then evaluated on two distances (20cm and 40cm) with three different ball types to assess both performance at the trained distance and generalization to a shorter distance and different objects.

C. Evaluation Setup

After training, the policy was deployed on the SO-101 follower arm for autonomous throwing evaluation. We conducted systematic testing at two distances (20cm and 40cm) with three different ball types.

Testing Protocol:

- 30 trials at 20 cm distance (per ball type)
- 30 trials at 40 cm distance (per ball type)
- Three ball types tested: pink ball, orange ball, rainbow ball
- Success criterion: Projectile lands completely inside basket
- Rim hits: Counted as failures (ball must land cleanly in basket)
- Same environmental setup as training (robot position, basket placement)

Evaluation Metric:

We evaluated performance using **success rate** (accuracy): the percentage of throws that successfully land in the target basket. This provides a direct, task-relevant measure of system performance. A throw is considered successful only if the projectile lands completely inside the basket without bouncing out.

D. Results

We evaluated the trained ACT policy across two target distances (20cm and 40cm) using three different ball types: pink ball, orange ball, and rainbow ball. Additionally, we used the pink ball training data as a baseline for comparison.

Table I summarizes the throwing accuracy achieved at each target distance and ball type.

TABLE I
THROWING ACCURACY RESULTS ACROSS DISTANCES AND BALL TYPES

Distance	Pink	Orange	Rainbow	Training*
20 cm	100%	96%	100%	75%
40 cm	50%	60%	46.67%	80%

*Training data = pink ball demonstrations used for policy training

Each distance was tested with 30 trials per ball type. A throw was counted as successful only if the ball landed completely inside the basket. Rim hits and bounces-out were counted as failures.

Key Findings:

- **Distance-dependent performance:** The robot achieved excellent accuracy at 20cm distance (96-100% across ball types) but significantly lower accuracy at 40cm distance (47-60%). This suggests the policy learned the throwing motion well but may require additional training data or refinement for longer distances.
- **Ball type impact:** At 40cm, the orange ball achieved the highest accuracy (60%), followed by the pink ball (50%) and rainbow ball (46.67%). The variation suggests that ball properties (weight, surface texture, aerodynamics) significantly affect throwing accuracy.
- **20cm performance:** Near-perfect accuracy (96-100%) at the shorter distance demonstrates the policy successfully learned smooth, coordinated throwing motions. The consistency across different ball types at this distance indicates robust performance.
- **Training data comparison:** Interestingly, the pink ball training data shows 75% accuracy at 20cm but 80% at 40cm. The robot achieved 100% at 20cm (surpassing training data) but only 50% at 40cm (below training data), suggesting the policy generalized very well for shorter throws but struggled with the longer distance.
- **Trajectory quality:** Visual observation confirmed smooth, coordinated throwing motions without jerky transitions. The ACT policy successfully produced natural-looking trajectories, validating the action chunking approach.
- **Release timing:** The policy learned consistent gripper release timing, critical for throwing accuracy. This demonstrates ACT's ability to capture temporal dependencies from human demonstrations.

E. Qualitative Observations

Beyond quantitative metrics, several qualitative observations emerged during testing:

- **Smooth motion:** The robot's throwing motions appeared natural and fluid, closely resembling the human demonstrations used for training.
- **Consistency:** Most throws followed similar trajectories, indicating the policy learned a stable throwing pattern rather than exhibiting random behavior.
- **Joint coordination:** The policy successfully coordinated multiple joints (shoulder, elbow, wrist) to produce effective throwing motions, validating the action chunking approach.
- **Failure modes:** When failures occurred, they were typically consistent (e.g., systematic bias in one direction) rather than random, suggesting potential for targeted improvement through additional demonstrations.

V. DISCUSSION

A. Analysis of Results

Our results demonstrate that imitation learning with ACT can successfully train a robot arm to perform dynamic throwing tasks. The results reveal interesting patterns in performance across distances and ball types.

Distance-Dependent Performance: The robot achieved excellent performance at 20cm (96-100% accuracy) but moderate performance at 40cm (47-60% accuracy). This significant gap suggests that while the policy learned smooth throwing motions, it may have overfit to the 40cm training distance. Interestingly, the robot exceeded human training data performance at 20cm (100% vs 75% training accuracy), demonstrating successful generalization to shorter distances. However, at 40cm, robot performance (50-60%) fell below the training data (80%), indicating room for improvement at the trained distance.

Ball Type Effects: Different balls showed varying success rates, particularly at 40cm distance. The orange ball achieved the best performance (60%), while the rainbow ball had the lowest (46.67%). At 20cm, all balls performed near-perfectly (96-100%), suggesting that ball properties matter more for longer, more challenging throws. This highlights the importance of projectile selection in robotic throwing tasks.

Action Chunking Benefits: The smooth, natural-looking trajectories observed during testing validate ACT's action chunking approach. By predicting sequences of future actions rather than single timesteps, the policy avoided the jerky motions that plague standard behavioral cloning approaches. This temporal consistency was essential for the coordinated, dynamic motion required for throwing.

B. Comparison to Related Work

Our approach demonstrates several advantages compared to prior robotic throwing systems:

- **vs. TossingBot:** Our pure imitation learning approach is simpler, requiring no vision sensors or explicit physics models. While TossingBot may achieve higher accuracy through physics-based optimization, our method is more straightforward to implement and deploy.
- **vs. Kinematic Planning:** Unlike analytical approaches that require detailed system modeling and calibration, our learned policy adapts directly to the robot's dynamics through demonstration data.
- **Data Efficiency:** With 30 demonstrations for each distance, we achieved functional throwing behavior, demonstrating the data efficiency of the ACT architecture.

C. Limitations

Several limitations of our current system should be noted:

- **Fixed setup:** The policy was trained and tested with consistent robot positioning and basket placement. Generalization to novel configurations was not evaluated.
- **Single object type:** Training used a single projectile type. Generalization to objects with different weights, sizes, or aerodynamic properties is unknown.
- **Limited trials:** With 30 test trials for each distance, statistical confidence in the results is moderate. More extensive testing would provide stronger conclusions.
- **Projectile effects:** Ball bounce characteristics significantly impacted measured success rates, potentially underestimating the policy's actual throwing accuracy.

VI. FUTURE WORK

Building on our results, several directions for future development emerge:

A. Autonomous Ball Retrieval

Currently, the system requires manual projectile loading between throws. Integrating autonomous ball pickup would enable fully autonomous operation:

- Train a separate grasping policy for ball retrieval
- Combine throwing and grasping policies in a sequential pipeline
- Enable continuous autonomous throwing without human intervention

B. Multi-Object Generalization

Extending the system to handle different projectile types would increase practical utility:

- Collect demonstrations with various objects (different sizes, weights, shapes)
- Train a single policy to adapt throwing motion based on object properties
- Investigate whether vision-based object recognition could inform throwing strategy

C. Automatic Distance Adaptation

The current system requires training data for each target distance. Dynamic distance adaptation would be valuable:

- Incorporate target distance as an input to the policy
- Train on a continuous range of distances rather than discrete points
- Investigate whether the policy can extrapolate to distances outside the training distribution

D. Vision Integration

Adding visual perception could enable several enhancements:

- Visual target localization for throwing at arbitrary locations
- Feedback from camera observations to adjust throwing strategy
- Detection of basket position for automatic distance estimation

E. Performance Optimization

Several avenues exist for improving throwing accuracy:

- Collect additional targeted demonstrations for failure cases
- Implement online learning to fine-tune the policy using the robot's own throws
- Optimize projectile selection for minimal bounce
- Investigate physics-informed regularization to improve trajectory consistency

VII. CALIBRATION VIDEO

The calibration video demonstrating successful control of the SO-101 arm can be accessed at: `https://shorturl.at/BdgEn`

VIII. CONCLUSION

This paper presents our approach to training an SO-101 robot arm for accurate projectile throwing using imitation learning. By leveraging Action Chunking Transformers and leader-follower teleoperation, we aim to learn smooth, dynamic throwing motions without requiring explicit physics models. Our preliminary setup validates the feasibility of the approach, and we have established clear evaluation metrics and a structured timeline for completing the project. The coming weeks will focus on data collection, policy training, and comprehensive evaluation to demonstrate the effectiveness of imitation learning for dynamic manipulation tasks.

This project successfully demonstrated that imitation learning with Action Chunking Transformers can train a robot arm to perform dynamic projectile throwing tasks. Using leader-follower teleoperation, we collected 30 demonstrations at 40cm distance and trained an ACT policy that achieved 96-100% accuracy at 20cm and 47-60% accuracy at 40cm across different ball types.

Key accomplishments include:

- **End-to-end imitation learning pipeline:** We established a complete workflow from human demonstration collection through policy training to autonomous execution using the LeRobot framework.
- **Strong short-distance performance:** The robot achieved near-perfect accuracy (96-100%) at 20cm distance, demonstrating excellent generalization to shorter throws despite being trained at 40cm.
- **Smooth trajectory generation:** The ACT architecture successfully produced natural, coordinated throwing motions, validating the action chunking approach for dynamic manipulation tasks.
- **Multi-ball evaluation:** Testing with three different ball types revealed performance variations, with the orange ball achieving best results (60% at 40cm, 96% at 20cm).

Our results demonstrate the feasibility of learning complex dynamic skills through imitation learning on low-cost robotic platforms. While performance at the 40cm training distance (47-60%) indicates room for improvement, the excellent generalization to 20cm throws (96-100%) and the consistently smooth trajectories validate the overall approach.

The methodology developed here is transferable to other dynamic manipulation tasks requiring coordinated, time-dependent control. Future work on additional training data at longer distances, multi-object generalization, and vision integration could significantly enhance the system's capabilities and practical utility.

This project contributes to the growing body of work showing that modern imitation learning techniques, combined with appropriate architectural choices like action chunking, can enable robots to acquire sophisticated motor skills through data-driven learning rather than explicit programming or physics modeling.

REFERENCES

[1] Hugging Face, "LeRobot: State-of-the-art Machine Learning for Real-World Robotics," 2024. [Online]. Available: `https://github.com/huggingface/lerobot`

[2] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, "TossingBot: Learning to Throw Arbitrary Objects with Residual Physics," *IEEE Transactions on Robotics*, 2020, arXiv:1903.11239 [cs.RO]. [Online]. Available: `https://arxiv.org/abs/1903.11239`

[3] donutsorelse, "Candy Throwing Robot Arm," Instructables, Aug. 2025. [Online]. Available: `https://www.instructables.com/Candy-Throwing-Robot-Arm/`

[4] J. McKay, "Penn State Altoona - Robotic Basketball Arm," 2020. [Online]. Available: `https://www.a-mc.com/pennstate-robotic-basketball-arm/`

[5] D. Son and S. Park, "LiPo: A Lightweight Post-optimization Framework for Smoothing Action Chunks Generated by Learned Policies," June 2025, doi: 10.48550/arXiv.2506.05165.

[6] M. Zare, P. M. Kebria, A. Khosravi, and S. Nahavandi, "A Survey of Imitation Learning: Algorithms, Recent Developments, and Challenges," arXiv preprint arXiv:2309.02473, 2023. [Online]. Available: `https://arxiv.org/abs/2309.02473`

[7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," arXiv preprint arXiv:2304.13705, 2023.